

Layers of Product Design — Session Log

Design decisions surfaced using the *Layers of Product Design* framework (seven layers, three zones). Each section records **decisions made**, **decisions uncovered**, and **open questions** — not just diagrams.

Diagrams are Mermaid and render natively in VSCode with the [Markdown Preview Mermaid Support](#) extension.

Convention — append only

This log is append-only. Never overwrite or replace a previous session.

Each `/layers-*` run adds a new dated `## Session N` section at the end. Earlier sessions stay exactly as written, including decisions later reversed and claims later corrected — the record of how thinking changed is itself the value. When a later session contradicts an earlier one, add a *Corrections* subsection to the new session (see Session 2) rather than editing the old one. When a decision is un-made, mark it **HELD** in place (see D1) rather than deleting it.

Documents in this folder

File	Purpose	PDF export
<code>layers-session.md</code>	This log. Design rationale and decisions, all sessions.	— (too long to work from live)
<code>moderator-engineers.md</code>	Protocol A — live script, engineer sessions	<code>moderator-engineers.pdf</code>
<code>moderator-experts.md</code>	Protocol B — live script, expert sessions	<code>moderator-experts.pdf</code>
<code>capture-sheet.md</code>	One copy per participant, filled in live	<code>capture-sheet.pdf</code>

The markdown is the source of truth. The PDFs are exports.

PDFs of the three operational docs are kept in this folder so the scripts and capture sheet can be printed, or lifted out of the project and used standalone during sessions without needing the repo open.

Consequence: they go stale silently. If a protocol changes, edit the markdown and re-export — never edit a PDF, and never treat one as authoritative when it disagrees with its `.md`. Exports are from 2026-07-28.

Sessions so far

#	Skill	Layer	Outcome
1	<code>/layers-orient</code>	all seven	Bottleneck identified: conceptual model
2	<code>/layers-conceptual-model</code>	conceptual model	8 decisions (D1 held), object model, ubiquitous language

#	Skill	Layer	Outcome
3	/layers-observed-behaviour	observed behaviour	Study plan, 2 protocols, 4 research questions

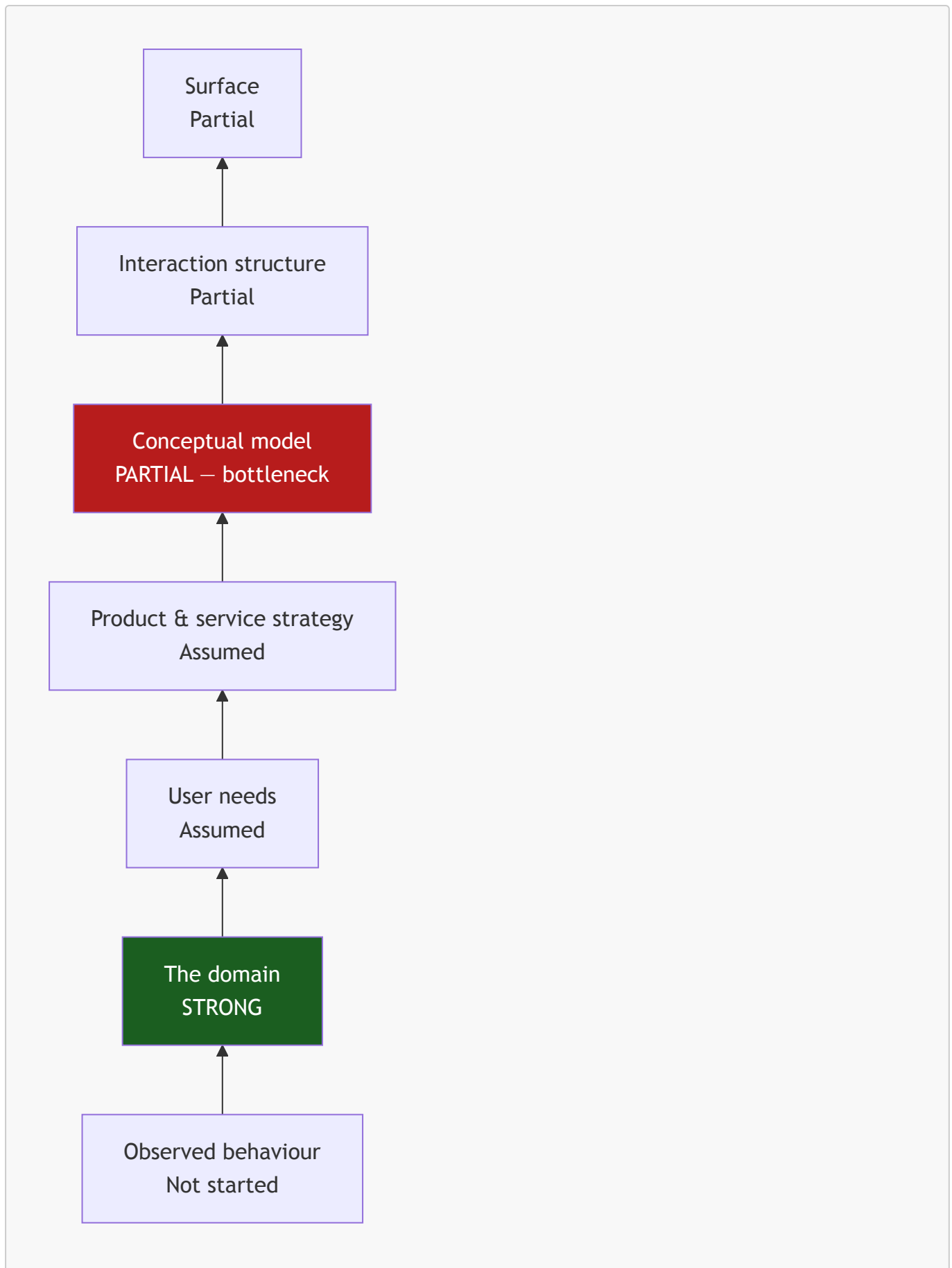
Session 1 — /layers-orient (2026-07-28)

Rapid diagnostic across all seven layers to locate the bottleneck.

Context given: OrchestratoR is a published portfolio POC, not a product seeking market fit. No user research exists — the design rests on the author's own accessibility judgement. The trigger for this session was sharpening the thinking so the repo holds up to a careful reader.

Decision landscape

Layer	State	Notes
Observed behaviour	Not started	No research. Own judgement. A deliberate choice for a POC, now stated rather than implied.
The domain	Strong	WCAG 2.1, W3C/WAI, DOJ ADA Web Rule named as source of truth; nine rules traceable to real criteria; rag/ registry files. The one solid foundation.
User needs	Assumed	No job stories. Target user differs across three documents.
Product & service strategy	Assumed	"Issues caught upstream save hours downstream" is a business-outcome claim never connected to a specific need or measurement.
Conceptual model	Partial — bottleneck	Objects exist in the type system, but three unresolved tensions (below).
Interaction structure	Partial	Happy path specified and built. Temporal states (empty / loading / partial / failure) thin.
Surface	Partial	Built, light and dark. No named design system; explicit "avoid over-design" constraint.



Decisions uncovered

Three at the conceptual model layer, all visible in the type system:

1. Two verdict vocabularies that do not reconcile. `RuleStatus` is `Pass | Unknown | Fail` (`types/evaluation.ts:1`). `A11yDoubleCheckResult.status` is `PASS | PARTIAL | FAIL (:10)`. Different casing, and the middle terms are different concepts — `Unknown` means *no evidence available*, `PARTIAL` means *partially compliant*. Three separate quality signals coexist (`healthScore`, `confidenceScore`, `shipReadiness`) with no stated relationship, and the latter two are typed as bare `string`, so their vocabularies are not pinned anywhere. → OOUIX **masked** failure mode: two distinct verdict objects presented as one.

2. What is the health score a property of? `Unknown` is excluded from the score denominator (`buttonEvaluator.ts:167-172`), so a component with 1 pass / 0 fail / 8 unknown scores **100**. The seeded sample also scores 85 in light mode and 100 in dark. The score is therefore a property of a *component × theme × evidence-available* triple, not of a component — but the model does not say so and the panel shows one number.

3. No evaluation identity or history. `EvaluationResult` has no id, timestamp, or component reference. Whether an evaluation persists, whether "did this improve?" is answerable, and what a Component is as an object (currently: pasted text) are all undecided.

Flagged — cross-cutting

Target user is inconsistent across artefacts:

- "design, product, engineering, and compliance" — [README.md:5](#)
- "developers and engineers" — [orchestrator_wireframe_spec.md:9](#)
- "a designer or engineer" — [orchestrator-architecture-diagram.md:59](#)

A user-needs wobble that propagates upward into what the panel should say and who the fixes are written for.

Bottleneck analysis

Conceptual model. Strictly, the framework points at the lowest weak layer, which is observed behaviour. That was overridden deliberately: this is a portfolio POC, so user research is the wrong investment, and the conceptual model carries *risky decisions already made* rather than decisions consciously skipped — a more urgent category. The tradeoff is named rather than ignored.

Open questions

- Do `Unknown` and `PARTIAL` collapse into one vocabulary, or are they genuinely different objects that need distinct visual treatment?
- Is the health score per-theme, or is there a single component score with theme as a dimension beneath it?
- Should an Evaluation persist? (Design question first; storage follows.)
- Which single audience do the fixes address?

Next

`/layers-conceptual-model` — resolve the verdict vocabulary clash, pin what the score is a property of, and settle Component and Evaluation as objects.

Session 2 — /layers-conceptual-model (2026-07-28)

Scoped deliberately: verdict vocabulary and score semantics as the real work, persistence as a stated decision rather than a build, audience settled inline. Method: walking the existing product (the implicit model in the code), since this is a redesign rather than a greenfield definition.

Corrections to Session 1

Two claims from the orient audit were overstated once the components were read, not just the types:

1. **Identity and time do exist.** `evaluationSignature` with staleness detection (`OrchestratorPanel.tsx:53-55`) and `approvedAt` with an approver (`:57`) are both present. True of the `EvaluationResult` type, not of the app: both live in React component state.
2. **A fourth contradiction, sharper than the original three** — findings ordering, below.

Decisions made

D1 — Two verdict vocabularies, deliberately distinct (not merged). *Status: HELD, not ratified.* The rename below assumes the `DoubleCheck` verdict is about whether the `Evaluation` was correct. That reading came from the architecture diagram's "second-pass validation" and was never checked against the prompt template that produces the verdict. If the verdict is instead about the component's accessibility, the two vocabularies are the same kind of thing and should be merged rather than renamed — the opposite conclusion. Verify before implementing.

They were never the same kind of thing, which is why they resisted reconciliation:

- `Pass` | `Fail` | `Unknown` — a verdict about **the component's compliance**
- `PASS` | `PARTIAL` | `FAIL` — a verdict about **whether the first pass was correct**, i.e. agreement, not compliance

Renaming the second removes the OOUX *masked* failure mode:

`Confirmed` | `Partially confirmed` | `Disputed`

`Unknown` (no evidence available) and `Partially confirmed` (evidence exists, agreement is partial) are now unmistakably different concepts.

D2 — `shipReadiness` becomes an enum, not prose.

```
ShipReadiness = "Ship ready" | "Ship with caution" | "Not ship ready"
```

Approval is currently gated on `shipReadiness.toLowerCase().includes("ship ready")` (`OrchestratorPanel.tsx:65`) — string-sniffing model prose. If the model returns "ready to ship", the approval affordance silently never appears. An enum makes the gate an equality check. Same applies to `confidenceScore`, also typed as bare `string`.

D3 — The health score belongs to an Evaluation, and an Evaluation is scoped by a RenderContext.

This is the object that was missing, and it explains why one component reports 85 in light mode and 100 in dark. A score is never a property of a Component alone.

The Component's headline score is the **worst-case across render contexts**, not an average: accessibility is a floor, not a mean. A component that fails in light mode is not 92% accessible — it is inaccessible in light mode.

D4 — Score and coverage are a pair and never appear apart.

Unknown is excluded from the score denominator ([buttonEvaluator.ts:167-172](#)), so 1 pass / 0 fail / 8 unknown scores 100. The formula stays — it is honest and documented — but the model now requires the score to carry its evidence coverage: **coverage = (pass + fail) / total**. Displayed as "100 · 1 of 9 rules verified", never as "100".

D5 — Findings order is Fail first. The policy doc is wrong, the code is right.

Three artefacts currently disagree:

Artefact	Order
mvp_scoring_policy.md:19-23 — "canonical across all documentation"	Pass → Unknown → Fail
rules.ts:9 , applied at buttonEvaluator.ts:577	Pass → Unknown → Fail
OrchestratorPanel.tsx:68-74	Fail → Unknown → Pass

The evaluator sorts one way and the panel immediately reverses it. Both ship.

Decision: **Fail first**. A panel whose purpose is surfacing problems must lead with them. Update the policy doc and **ORDERED_FINDING_STATUS**; remove the double sort so one ordering exists in one place.

D6 — Approval is scoped to an evaluation signature and voided by any change.

The existing reset-on-change behaviour ([:59-61](#)) is correct and is now a stated decision rather than an implementation detail. Approval attaches to a specific evaluated state, never to a Component in general — otherwise a component could carry approval it no longer earns.

D7 — Audience: engineers. Proposed, pending confirmation. The fixes are already written in engineering language and the entire surface is IDE-shaped. Resolves the three-way conflict flagged in Session 1; README, wireframe spec, and architecture diagram should be aligned to it.

D8 — Evaluations do not persist. Decided, not built. A POC evaluates the component in front of you; there is no history, no trend, no "did this improve?". Stated so the absence reads as scope, not oversight.

Object definitions

Object: Component

What it is:

accessibility.

Attributes:

Relationships:

Actions:

Note:

A UI element a developer wants checked for

evaluated by zero or many Evaluations

Load component code

Currently pasted text, not a persisted object (see D8).

Object: RenderContext

What it is: The visual conditions a component is judged under.
 Attributes: theme (light | dark), interaction state (default | hover | active | disabled | focused)
 Relationships: scopes zero or many Evaluations
 Actions: Switch theme, Select state

Object: Rule

What it is: One accessibility requirement the product checks.
 Attributes: id, name
 Relationships: traces to exactly one Criterion; produces many Findings
 Actions: (none – product-defined, not user-editable)
 Instances: 9 – semantic markup, accessible name, keyboard operability, focus visibility, text contrast, focus indicator contrast, target size, disabled state behaviour, state coverage

Object: Criterion

What it is: The external standard a Rule derives its authority from.
 Attributes: reference, publisher (WCAG 2.1 | W3C/WAI | DOJ ADA Web Rule)
 Relationships: referenced by one or many Rules
 Note: `sourceReference` currently duplicated as a string attribute
 is on both Rule and Finding – an attribute that is really a relationship. Modelling it makes the traceability that this product's strongest asset explicit.

Object: Finding

What it is: What one Rule concluded about one Component in one context.
 Attributes: status (Pass | Fail | Unknown), evidence, severity, recommendation
 Relationships: belongs to one Evaluation; produced by one Rule
 Actions: View source lines, Copy hex value

Object: Evaluation

What it is: One complete run of all Rules against a Component.
 Attributes: health score, coverage, summary, scoring policy label, signature
 Relationships: of one Component; scoped by one RenderContext; contains one or many Findings; may be verified by zero or one DoubleCheck
 Actions: Run accessibility check

Object: DoubleCheck

What it is: A second-pass review of whether an Evaluation got it right.
 Attributes: agreement (Confirmed | Partially confirmed | Disputed), confidence, ship readiness, evidence summary, remaining risks, recommended next step, verified items
 Relationships: verifies one Evaluation; may be approved by zero or one Approval

Actions: Run A11Y DoubleCheck
 States: current | stale (when the Evaluation signature changes)

Object: Approval

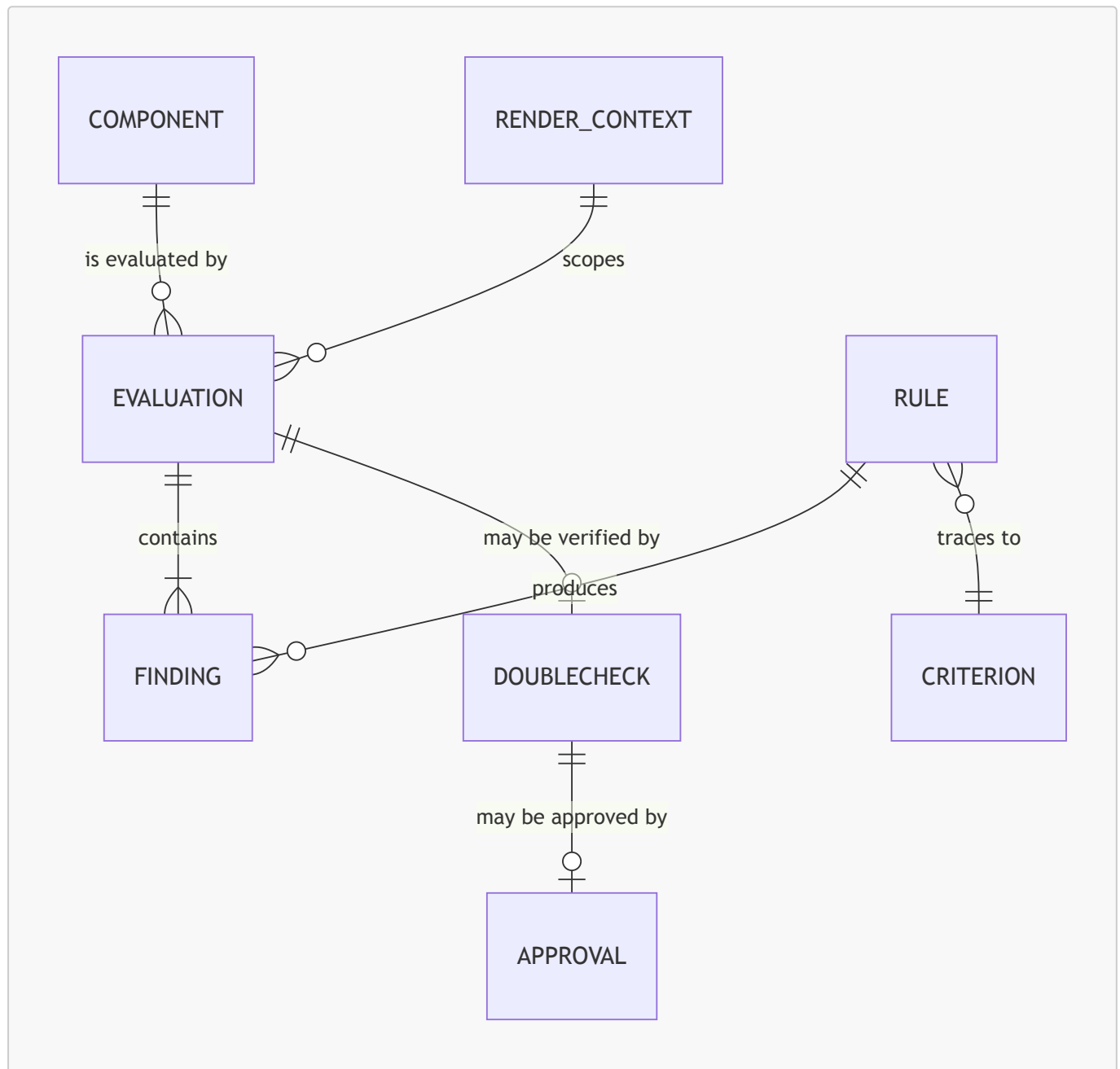
What it is: A human's recorded decision that this is fit to ship.

Attributes: approver, approved at

Relationships: approves one DoubleCheck

Actions: Approve for ship

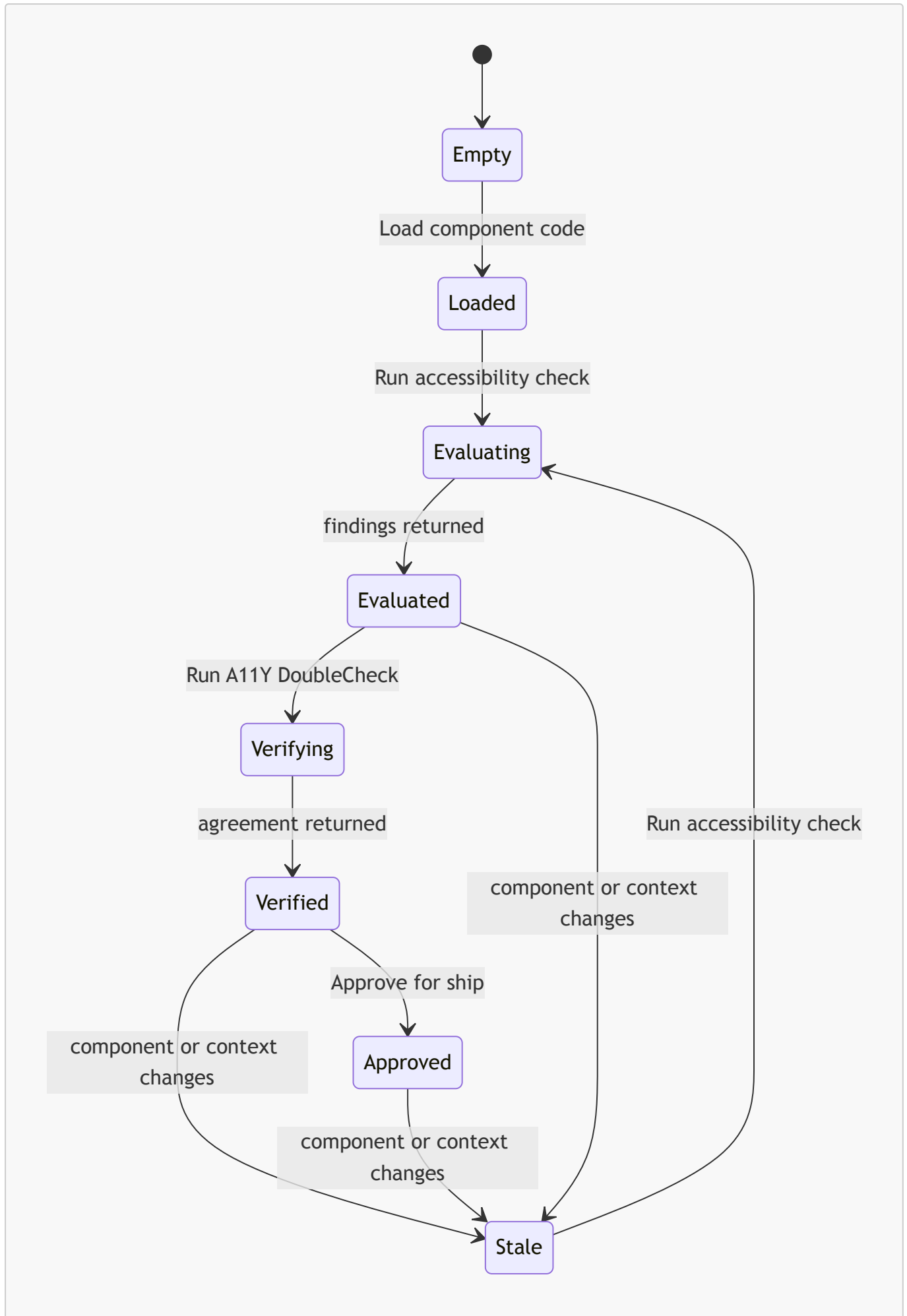
Object map



Cardinality: || exactly one · o{ zero or many · |{ one or many. The crow's foot sits on the many side.

State transitions

The Evaluation lifecycle, including the staleness path that already exists in the code:



What becomes impossible where: **Approve for ship** is unavailable unless agreement is **Confirmed**, ship readiness is **Ship ready**, and remaining risks are empty. Reaching **Stale** voids any Approval (D6).

Ubiquitous language

Nouns

Term	Rejected alternatives	Decision
Finding	RuleResult	User-facing word; "result" is engineering framing. Already the section heading in the panel.
Evaluation	Check, Scan, Run	"Check" is the verb; the noun should differ from it.
DoubleCheck	Second pass, Validation, Verification	Established product name with a trademark-adjacent identity. Keep.
RenderContext	Theme, Mode, Variant	"Theme" covers only light/dark; interaction state belongs to the same object.
Criterion	Source reference, Standard	Matches WCAG's own vocabulary — success criterion.
Accessibility	A11Y, A11y	Three spellings currently ship: "Accessibility Co-pilot" (panel:152), "A11Y DoubleCheck" (:159), "A11y Inspector" (wireframe spec:71). Use "Accessibility" in prose; keep "A11Y DoubleCheck" as the one proper noun.

Verbs

Verb	Applies to	Rejected	Decision
Run	Evaluation, DoubleCheck	Start, Execute, Check	One verb, used identically for both passes. Learn once.
Load	Component	Paste, Import, Upload	Names what the user gets, not the mechanism.
Approve for ship	Approval	Approve Component	Verb/object mismatch corrected: the UI says "Approve Component" (:291) but what is approved is a specific evaluated state, not the component in general (D6). The internal name canApproveForShip was already right.
Ask	—	Query, Search	Conversational affordance, distinct from Run.

Flattening check. No generic CRUD verb here hides operations with different real-world consequences — the product is read-only over the component. The one genuine flattening risk was Approve, resolved above: approving a component and approving an evaluation have different consequences when the component changes.

Open questions

- **Does a Component score exist at all, or only per-context Evaluations?** D3 says worst-case, but that assumes the UI shows one headline number. If both themes are shown side by side, the aggregate may be unnecessary. Interaction layer decides.
- **What is **confidence** measuring?** Distinct from coverage (D4) and from agreement (D1), but the difference isn't yet articulated. Risk of a third redundant quality signal. Flagged rather than resolved.
- **Does Criterion earn its place in a POC?** Correct by OOUX and it makes traceability explicit, but adds an object for a POC that evaluates one component type. Defer to whether traceability becomes user-facing.
- **Should **Disputed** block approval outright?** Currently only **PASS** permits it, so **Disputed** blocks by omission. Whether that should be explicit and explained to the user is an interaction-layer decision.
- **Confirm D7 (audience: engineers).** Proposed, not ratified.

Caveat

Built without domain research — Session 1 recorded Observed behaviour as Not started. This model is a hypothesis grounded in the existing implementation and in WCAG as an external standard, not in evidence about how developers actually work. The vocabulary decisions are safe (they resolve internal contradictions); the object boundaries are the parts most likely to be wrong.

Next

/layers-interaction-flow — design how users move through these objects, including the temporal states the audit rated thin (empty, loading, partial, failure paths).

Session 3 — **/layers-observed-behaviour** (2026-07-28) — Plan mode

Study design for the first research Orchestrator has had. Session 1 rated this layer Not started; this plan closes that gap and, deliberately, tests three decisions left unrati ed in Session 2.

Format: moderated, live, screen share, think-aloud. 30 minutes. **Segments:** accessibility experts + engineers ("to start with" — wave 1).

Research questions

#	Question	Segment	Method
RQ1	When a developer hits an accessibility problem in a component today, what triggers them to act, what do they reach for, and where does it stall?	Engineers	JTBD retrospective (compressed — see caveat)
RQ2	Can someone unfamiliar with OrchestratoR load a component, run an evaluation, and reach a ship / don't-ship decision unaided — and where do they hesitate?	Both	Usability observation, think-aloud
RQ3	What do people believe the health score, Unknown , and the DoubleCheck verdict <i>mean</i> — and does that match what the model intends?	Engineers (primary)	Comprehension probes
RQ4	Are the nine rules and their verdicts defensible to someone who does accessibility professionally?	Experts only	Expert review probes

RQ1 caveat. At 30 minutes RQ1 gets ~6 minutes: one grounded story, not saturation. Findings on it are directional and must be marked **inferred**, never **observed**. A real answer to RQ1 needs its own study.

Why the segments are not pooled

Accessibility experts already know what **Unknown** means and what WCAG 1.4.3 requires. Their comprehension answers are therefore **not evidence** about target-user comprehension — using them that way would manufacture a false pass on RQ3. What experts uniquely provide is judgement on whether the findings are *correct* (RQ4), which engineers cannot give.

- **Engineers** → RQ1, RQ2, RQ3. The target user per D7.
- **Experts** → RQ2, RQ4. Content validity and credibility.

Tag every observation with the segment it came from. Do not aggregate RQ3 across both.

Sample

Wave 1: **4 engineers + 3 accessibility experts = 7 sessions.**

Engineers should build UI components as part of their actual job and should *not* be accessibility specialists — the comprehension data depends on them being representative of the target user. Recruit for range in prior WCAG exposure rather than screening it out; "has heard of contrast ratios but couldn't quote a threshold" is the modal target user.

Below 4 engineers, RQ3 stays directional. That is an acceptable wave-1 outcome if recruitment is hard — state it rather than over-claiming.

Two separate protocols

The segments are interviewed separately, so each gets a purpose-built 30 minutes rather than one script with skipped blocks. The expert session spends the time engineers give to RQ1 on content validity instead.

Shared opening for both (0–3 min · Setup): consent to record. Frame honestly — *"This is a proof of concept I built. I'm testing it, not you — if something is confusing, that's the finding I need."* Establish think-aloud. State that it evaluates buttons only.

Protocol A — Engineer session (30 min)

Covers RQ1, RQ2, RQ3. This is the comprehension and friction signal.

3–9 min · RQ1 — retrospective

Ground in a real past event, never a hypothetical:

"Tell me about the last time you shipped, or nearly shipped, a component with an accessibility problem. Walk me through what was happening."

Follow the story, don't lead it:

- "What made you notice?"
- "What did you reach for first?"
- "What did you do when that didn't settle it?"
- "Who else got involved, and when?"

Listen for **nouns** (candidate domain objects) and **workarounds** — a need real enough to motivate improvisation is the strongest signal available.

9–21 min · RQ2 — task, unaided

Task given: *"Find out whether this button is accessible enough to ship."* Nothing more. Do not mention the panel.

Do not help for the first 90 seconds of any stall. Silence is data.

Capture:

Observation	Why it matters
Did they find the panel trigger unaided, and how long did it take?	The panel is closed by default and opens from an icon in the top right (wireframe spec:61-65). If discovery fails, everything above it is unreachable.
Time to first completed evaluation	Baseline task-completion signal
Did they switch theme unprompted?	Gates the D3 probe below
Did they run DoubleCheck without being told it existed?	Tests whether the second pass is discoverable or decorative
Did they reach a ship / don't-ship decision, and on what basis?	The actual job. A participant who never reaches a verdict is the most important finding in the study.

Observation	Why it matters
Every point of hesitation, with what was on screen	RQ2's primary output

21–28 min · RQ3 / RQ4 — comprehension probes

Ask in this order. Never explain before they answer.

Score meaning (tests D4):

1. Point at the score. "What does this number tell you?"
2. "What would make it go up?"

Score subject (tests D3): if they didn't switch theme unprompted, do it for them now. 3. "Same button. Light mode says 85, dark says 100. What do you make of that?" 4. "So what's this component's score?"

Evidence gaps (tests D4): switch to the prepared no-CSS fixture, which returns a high score with most rules unverified. 5. "What does **Unknown** mean here?" 6. "Is **Unknown** better or worse than **Fail**?" 7. **"Would you ship this?"** — if yes, the bare number is actively misleading and D4 is load-bearing rather than pedantic.

Verdict subject (settles D1): 8. Point at the DoubleCheck verdict. "This says PASS. What is it saying passed?"

D1 was held in Session 2 because the rename assumed the verdict reports on whether the *Evaluation* was correct rather than on the *component*. This probe answers a better question than reading the prompt template: if users read **PASS** as "my button is fine", the verdict is functionally about the component whatever the prompt intends, and the two vocabularies should be merged rather than renamed.

Action (tests D6 / D7): 9. "What would you do next?" 10. If Approve is showing: "What does approving this mean? Who is it for?"

28–30 min · Close

"Is there anything I haven't asked that would help me understand this better?"

Optional stretch only if time remains: let them paste their own component. See the caveat below before interpreting anything from it.

Protocol B — Accessibility expert session (30 min)

Covers RQ2 and RQ4. **Not** a source of RQ3 evidence — see the segment note above. The six minutes engineers spend on RQ1 go to content validity here, plus a longer review block.

3–6 min · Current practice

Brief and factual, to calibrate their answers rather than to answer RQ1:

"When you audit a component today, what do you actually use?" "What does a finding have to include before you'd act on it?"

6–15 min · RQ2 — task, unaided

Same task and same rules as Protocol A: *"Find out whether this button is accessible enough to ship."* No mention of the panel, no help for 90 seconds.

Experts will complete this faster than engineers, which is expected. What matters here is *where an expert hesitates anyway* — a stall for someone who knows the domain cold is a pure interface problem, with subject-matter confusion ruled out. That makes expert friction data unusually clean, and worth weighting heavily on RQ2 even though it is useless for RQ3.

15–27 min · RQ4 — content validity

Walk three or four findings, including at least one **Fail** and one **Unknown**:

1. "Is this finding correct?"
2. "Is the evidence it gives sufficient to justify that verdict?"
3. "Is the recommended fix what you would advise?"
4. "Anything it should have caught and didn't?" — false negatives matter more than false positives for a tool claiming ship-readiness.
5. Show the rule list. "Are these the right nine? What's missing?"
6. "Would you trust this in a review? What would have to change first?"

Probe 4 is the one most likely to produce an uncomfortable finding, which is why it is scripted rather than left to time remaining.

27–30 min · Close

"Is there anything I haven't asked that would help me understand this better?"

Operational risks

1. The backend is a single point of failure. DoubleCheck, RAG query, and echo all require Flask running with a live **ANTHROPIC_API_KEY**. A failure mid-session destroys the RQ3 half with no recovery inside 30 minutes. Mitigation: pre-flight the endpoint before every session, and prepare static screenshots of a completed DoubleCheck so probes 8–10 can be run verbally as a fallback.

2. Own-component paste will confound the **Unknown data.** CSS is located by matching **.btn**, falling back to **button**, so a real-world component may under-analyse and return **Unknown** rows that look like evidence gaps but are selector misses. Keep the scripted task on the seeded sample. Discount any **Unknown** observed on a pasted component, and note it in the record rather than silently dropping it.

3. Buttons-only is an artificial task. Name it in the framing so participants don't spend their think-aloud querying the scope instead of the interface.

Fixtures to prepare

1. **Seeded sample** — known good: 85 light / 100 dark. Used for the main task.

2. **No-CSS fixture** — TSX only, no stylesheet. Produces a high score with most rules unverifiable, which is what probes 5–7 need. Verify before session one that it actually returns a misleadingly high score; if the score lands low, the probe doesn't work.
3. **Static DoubleCheck screenshots** — light and dark, for backend fallback.

Synthesis plan

Agreed before collection, per framework guidance:

- One observation per note. Raw quote or observed action, never a summary.
- Tag each with: RQ number · segment (engineer / expert) · participant id.
- Mark confidence on every claim: **observed** (seen or heard directly) / **inferred** (reasonable reading of what was seen) / **assumed** (belief with no supporting observation).
- Flag workarounds separately — they are the highest-value signal.
- Keep a running tally per probe. Probe 7 ("would you ship this?") is a countable yes/no across engineers and is the single most decision-relevant number in the study.
- Do not draft job stories during collection. Draft after, in Synthesise mode.

Research gaps this study will not close

- **RQ1 at depth** — triggers and stalls get 6 minutes; no saturation.
- **Whether anyone would adopt this** — unanswerable by observation, since there is no adoption to observe. Deliberately out of scope.
- **Whether the tool helps** — no before/after comparison, so any efficiency claim remains unevicenced. The README's "hours saved" framing stays an assumption after this study.
- **Designers as a segment** — wave 1 is engineers and experts. D7 is therefore tested against engineers but not fully contested.
- **Repeat use over time** — a 30-minute first encounter says nothing about whether findings stay legible on the fiftieth run.

Next

Run the sessions, then return to [/layers-observed-behaviour](#) in **Synthesise mode** to extract observations, identify patterns, and draft candidate job stories with confidence ratings. Those feed [/layers-user-needs](#).